SCS 2312 – Take Home Assignment

Implementing a Parser

Introduction

parser.c is a command-line based parser and interpreter programmed in C language. It processes a small, custom-defined programming language that supports integer variable declarations, arithmetic expressions, and output statements.

It has following features:

- The tokeniser identifies and categorizes tokens into keywords, identifiers, operators, punctuation symbols, numeric literals, and unknown tokens.
- The parser is implemented using a recursive-descent approach, strictly following the defined CFG.
- The program executes valid declarations and print statements.
- The program detects and reports syntax and semantic errors with line and column numbers.
- Error recovery is implemented using panic-mode synchronization to continue parsing after errors.

CFG (Context Free Grammer)

<program> → <stmt_list>
<stmt_list> → <stmt> <stmt_list> | ε
<stmt> → <decl_stmt> | <print_stmt>
<decl_stmt> → int IDENT = <expr> ;
<print_stmt> → print (<expr>) ;
<expr> → <term> { (+ | -) <term> }
<term> → <factor> { (* | /) <factor> }
<factor> → NUMBER | IDENT | (<expr>)

Tokenization

Token Name	Lexeme / Pattern	Description
TOKEN_INT	int	Used for integer variable declarations
TOKEN_PRINT	print	Used to output the value of an expression
TOKEN_IDENT	[A-Za-z_][A-Za-z0-9_]*	Variable names consisting of letters, digits, and underscores
TOKEN_NUMBER	[0-9]+	Integer numeric literals
TOKEN_ASSIGN	=	Assignment operator
TOKEN_PLUS	+	Arithmetic addition operator
TOKEN_MINUS	-	Arithmetic subtraction operator
TOKEN_MUL	*	Arithmetic multiplication operator
TOKEN_DIV	/	Arithmetic division operator
TOKEN_SEMI	;	Statement terminator
TOKEN_LPAREN	(	Left parenthesis
TOKEN_RPAREN	)	Right parenthesis
TOKEN_EOF		Indicates end of source file
TOKEN_UNKNOWN	ex: @, #, $, %, ^, !	Any invalid or unsupported character

Code Explanation

CFG Rule / Component	**Implementation in parser.c**	**What it does**
<program>	parse_program(Parser *P)	Parses the whole file: loops until TOK_EOF, calling parse_stmt() for each statement.
<stmt>	parse_stmt(Parser *P)	Detects which statement starts next (int declaration or print). Also handles statement-level syntax checks + statement recovery.
<expr>	parse_expr(Parser *P)	Parses addition/subtraction
<term>	parse_term(Parser *P)	Parses multiplication/division
<factor>	parse_factor(Parser *P)	Parses literals/variables/parentheses: NUMBER or IDENT or (<expr>). Detects undeclared identifiers (semantic) + invalid expression start (syntax).
IDENT terminal	Lexer: lexer_next() producing TOK_IDENT	Recognizes identifiers: [A-Za-z_][A-Za-z0-9_]*

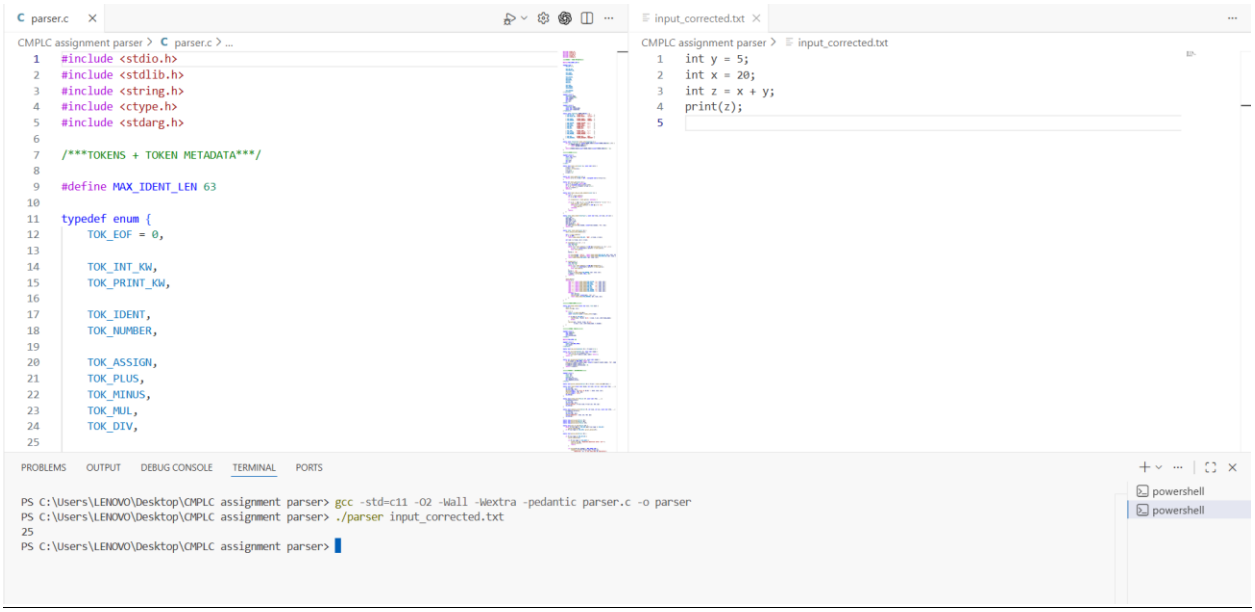
NUMBER terminal	Lexer: lexer_next() producing TOK_NUMBER	Recognizes integers: [0-9]+
Keywords / operators / punctuation terminals	Lexer: lexer_next() producing TOK_INT_KW, TOK_PRINT_KW, TOK_PLUS, etc.	Converts raw characters/words into token types for the parser.
Token metadata (pretty names)	TOKEN_TABLE[] + token_info()	Maps token type → printable enum name (TOKEN_INT) and expected text ('int') for debugging/token dump.
Syntax error reporting	syntax_error(Parser *P, ...)	Prints: Syntax error at line:col: ... and sets had_error.
Semantic error reporting (STRICT)	semantic_error(...) + flush_semantic_errors_if_no_syntax()	Stores semantic errors and prints them **only if** syntax_errors == 0.
Statement recovery	sync_to_semi(Parser *P)	Skips tokens until ; or EOF to continue parsing next statement.
Expression recovery	sync_to_semi_or_rparen(Parser *P)	Skips tokens until ; /) / EOF, preventing cascaded errors.
Symbol table (variable storage)	SymTable, sym_init, sym_find, sym_declare	Tracks declared variables and their values for execution + semantic checks.
File input	read_entire_file(path)	Loads source file into memory so lexer can scan it.
Entry point	main(argc, argv)	Handles modes: normal run (./parser file.txt) or token dump (./parser --tokens file.txt). Initializes parser + runs parse_program().

Program execution modes:

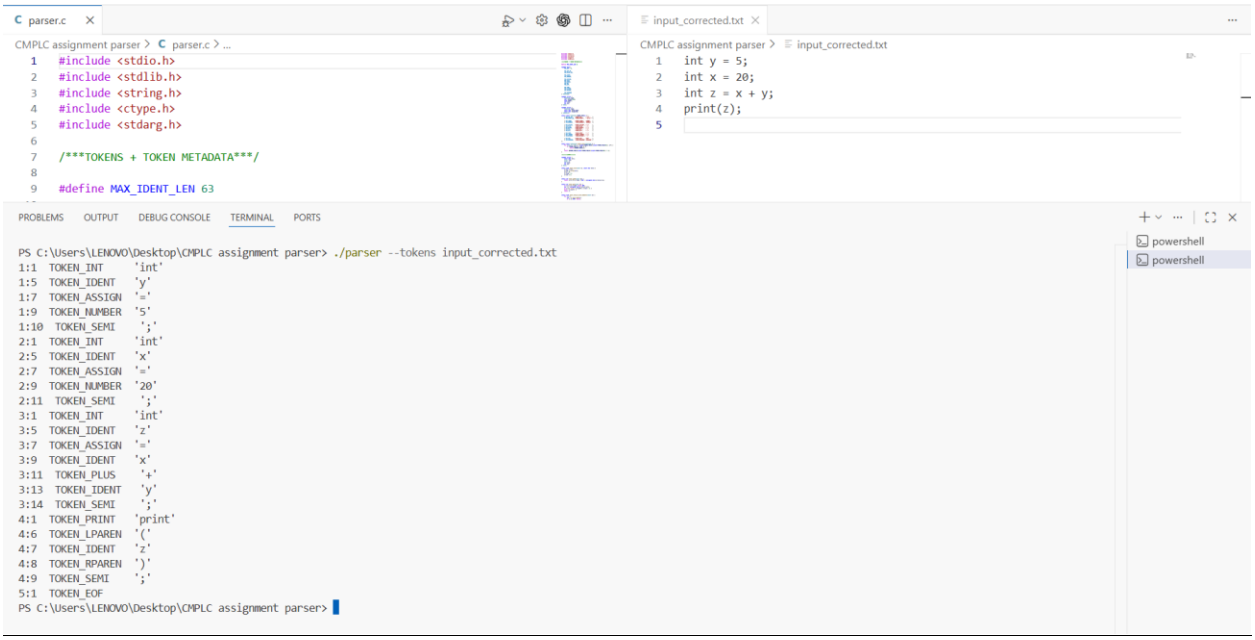
1) Parse and get the output of correct input file, and for error detection in programs with syntax and semantic errors: **./parser <inputfile.txt >**

2) To derive the tokens of the program: **./parser –tokens <inputfile.txt>**

Correct Input

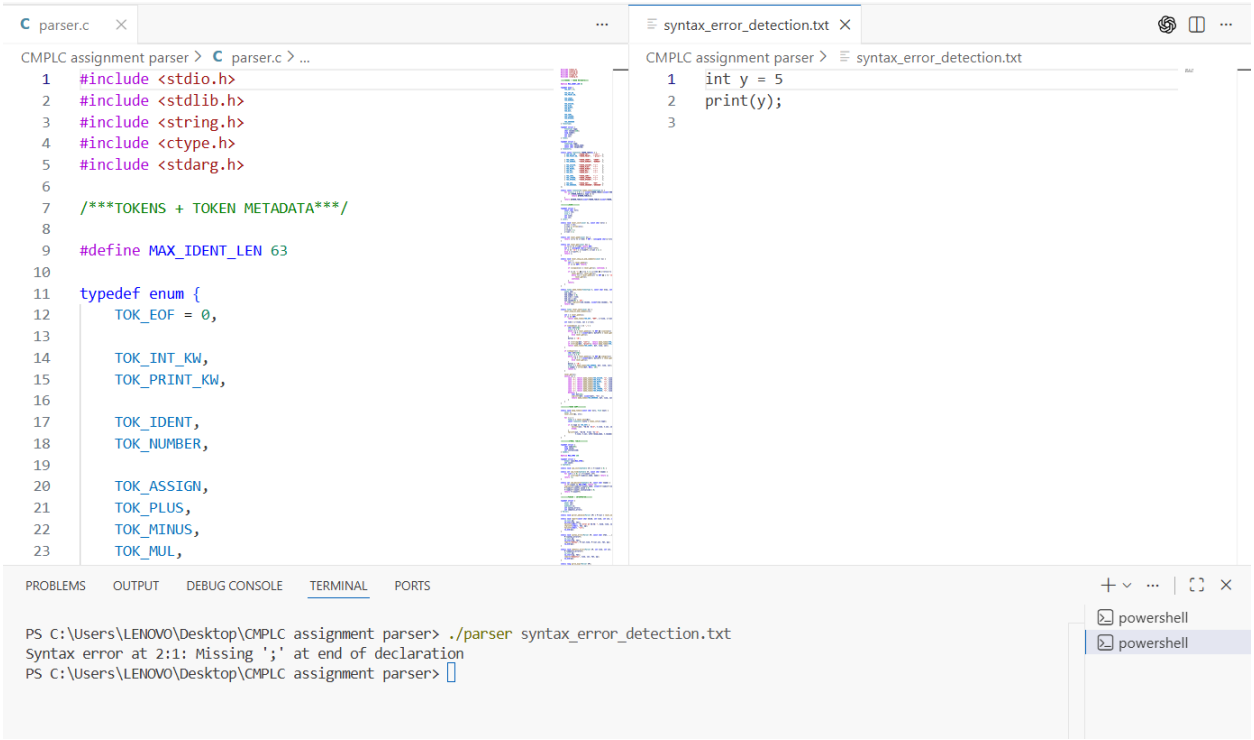


Printing Tokens

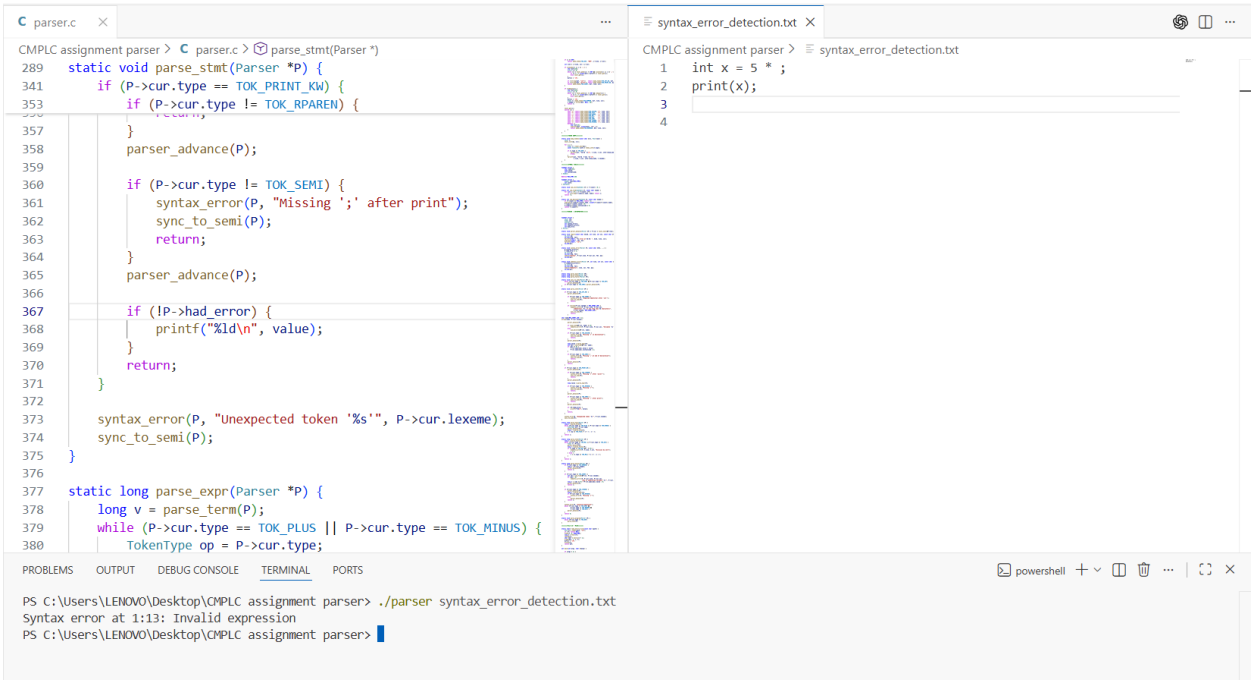


Detecting Errors

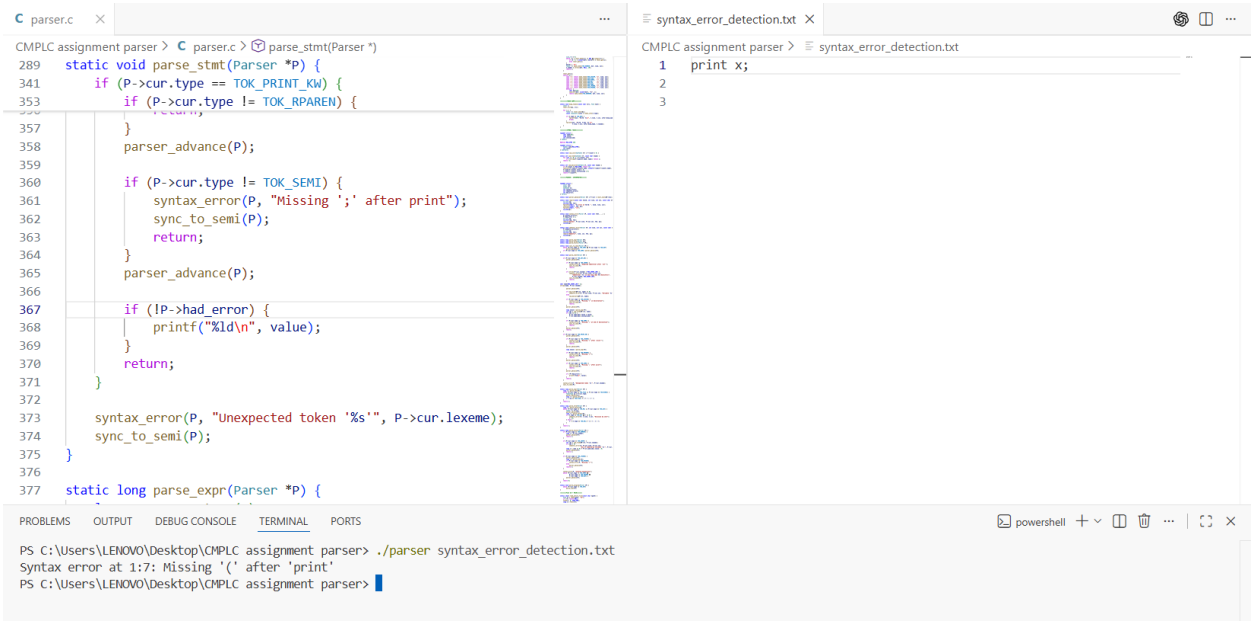
1) Syntax errors
   - Missing semicolon

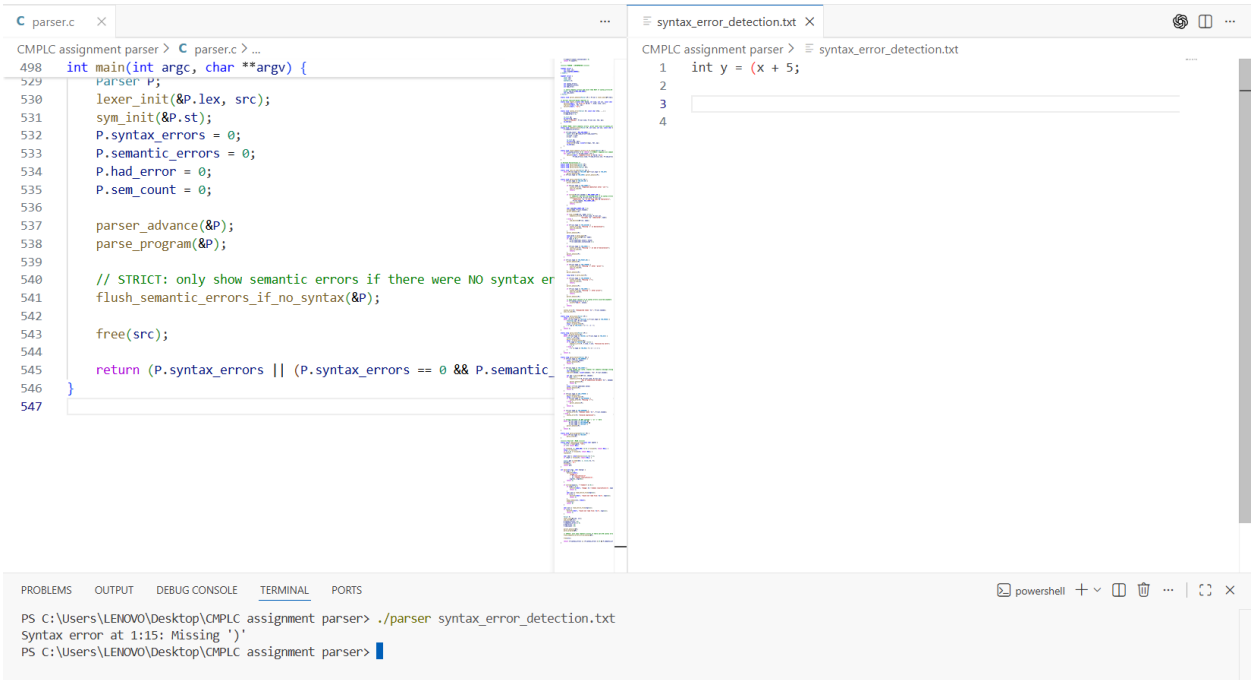


   - Invalid expression

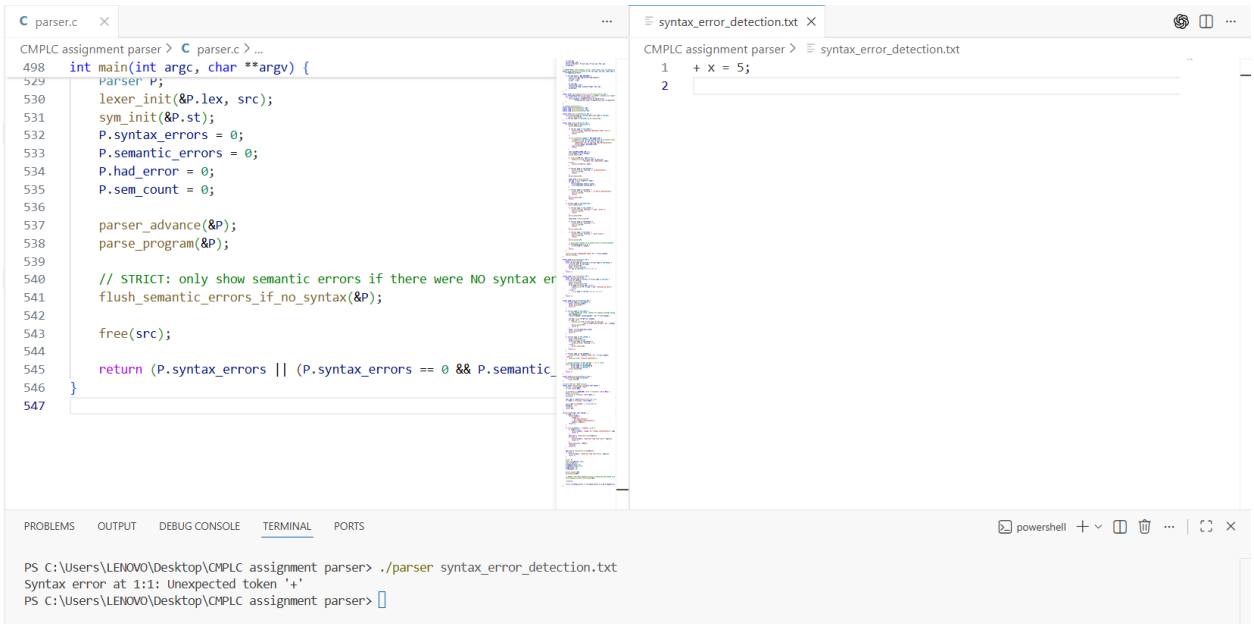


- Missing parenthesis

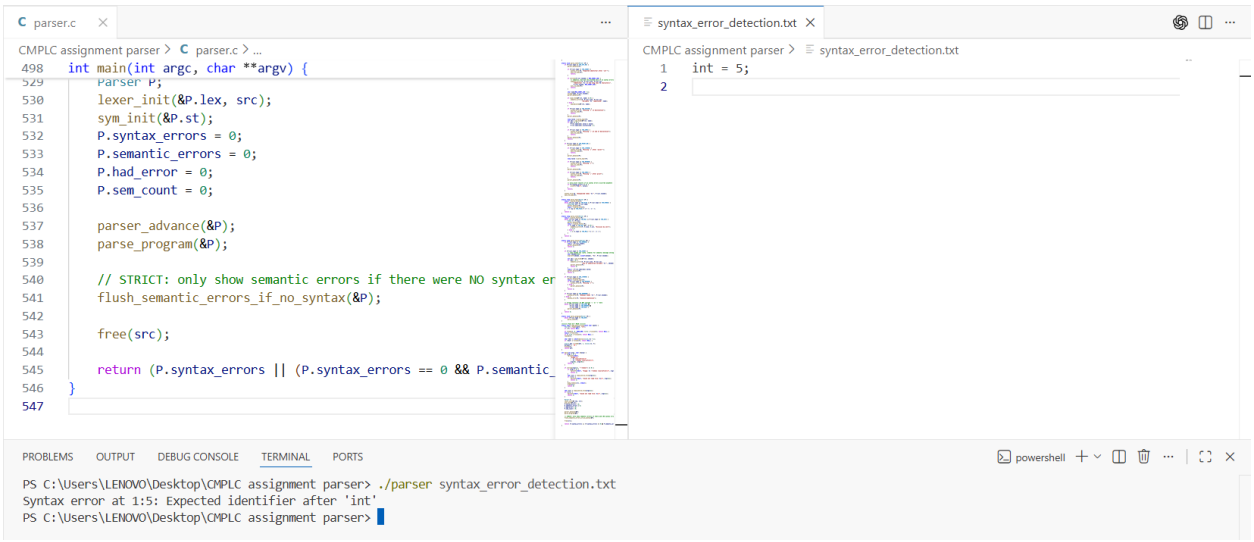




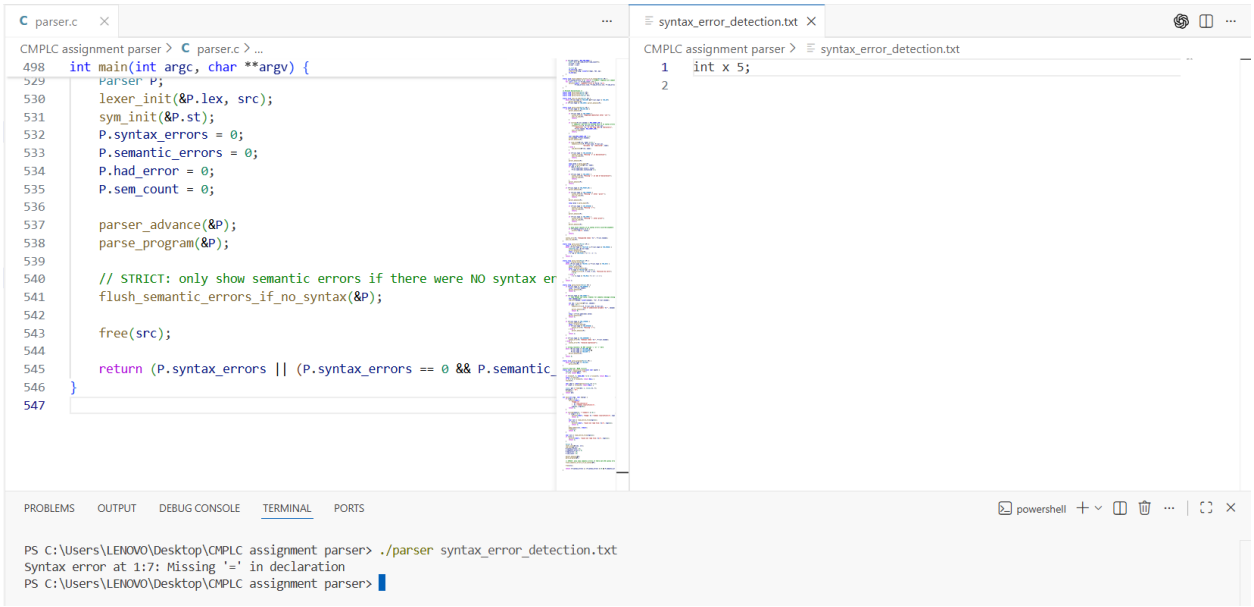
- Unexpected statement start



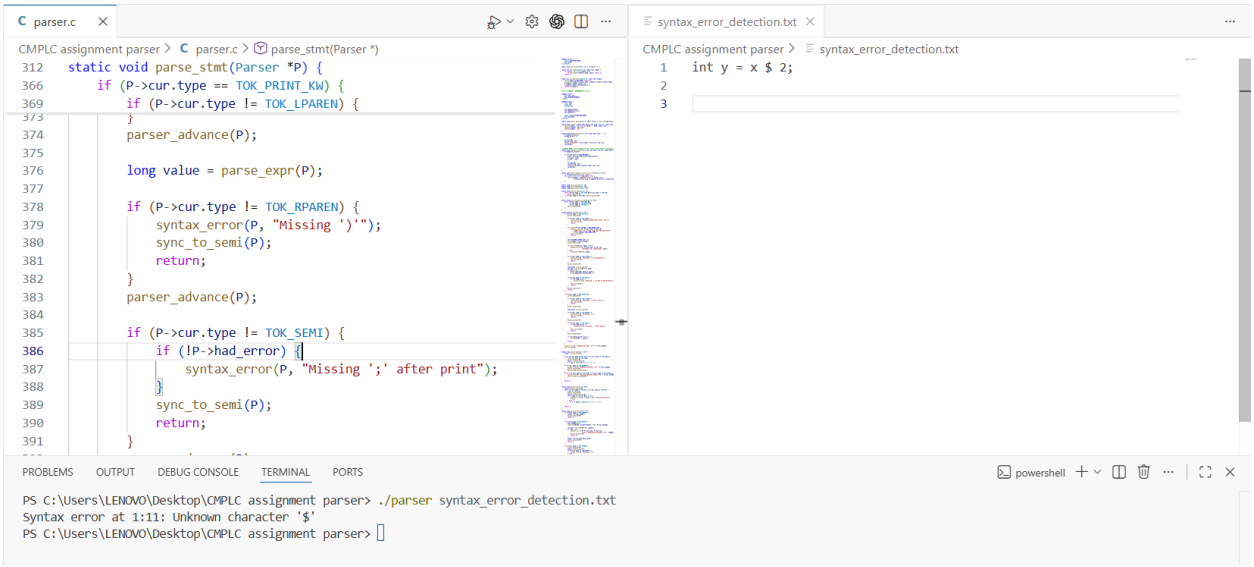
- Missing Identifier



- Missing Assignment operator in declaration

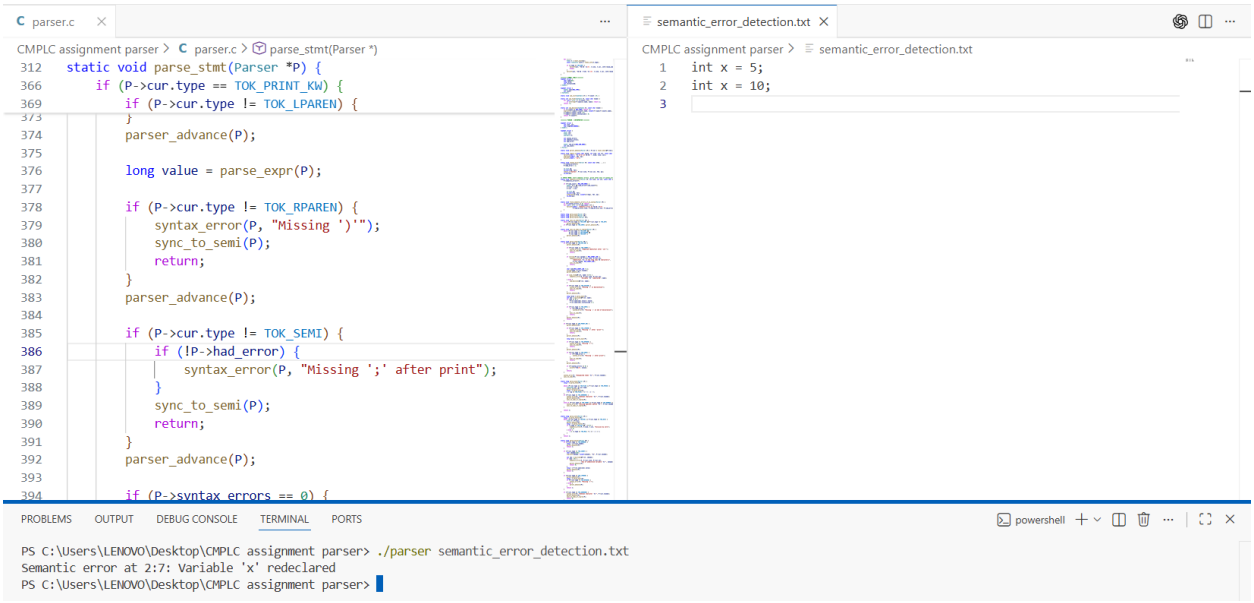


- Unknown / Invalid Character



2) Semantic Errors

- Redeclaration of Variables



- Undeclared variables

